Neural Network
Back Propagation Training Algorithm

CSAI 253 - Spring 2024

Prepared By

Dr. Khaled Mostafa ElSayed

kmelsayed@zewailcity.edu.eg

Instructors:

Dr. Khaled Mostafa ElSayed

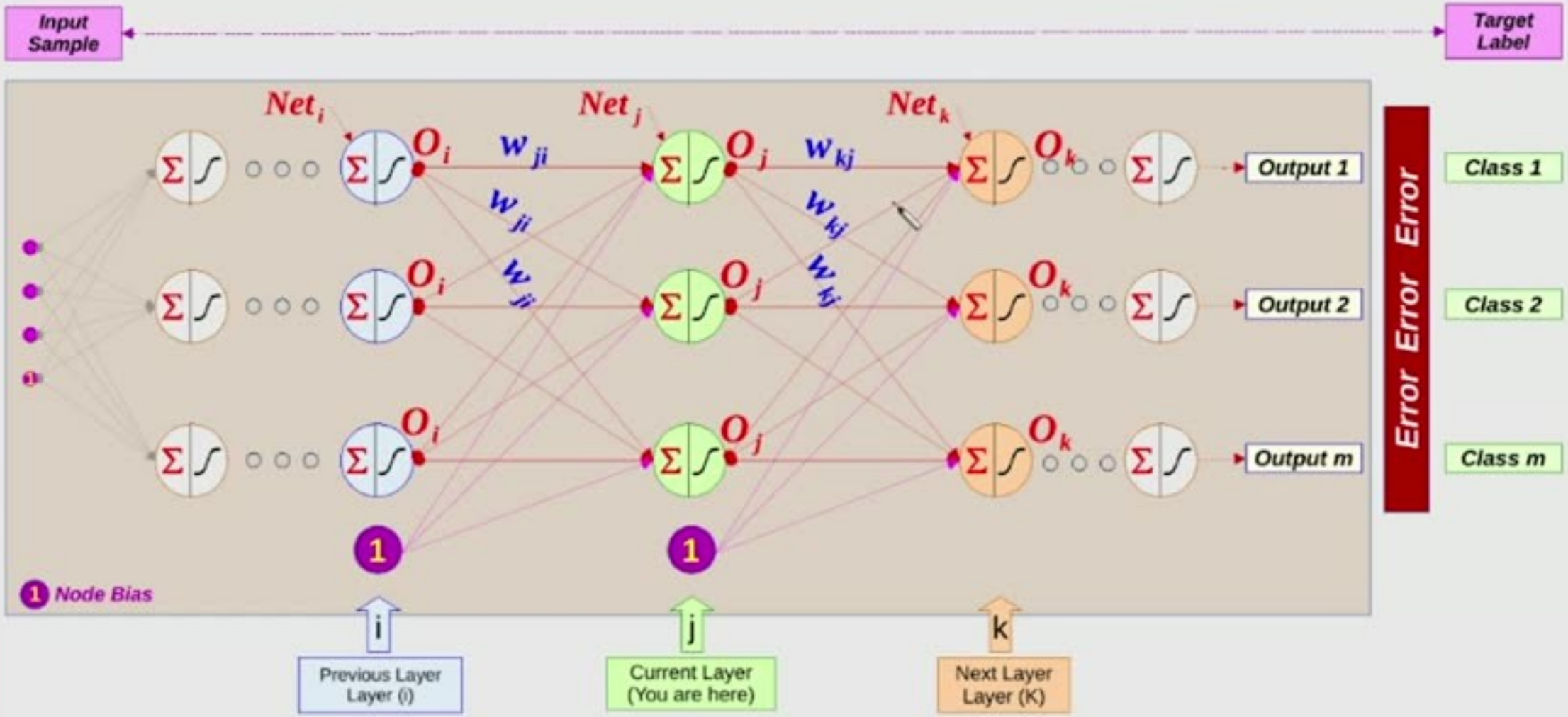
kmelsayed@zewailcity.edu.eg

Dr. Mohamed Maher Ata

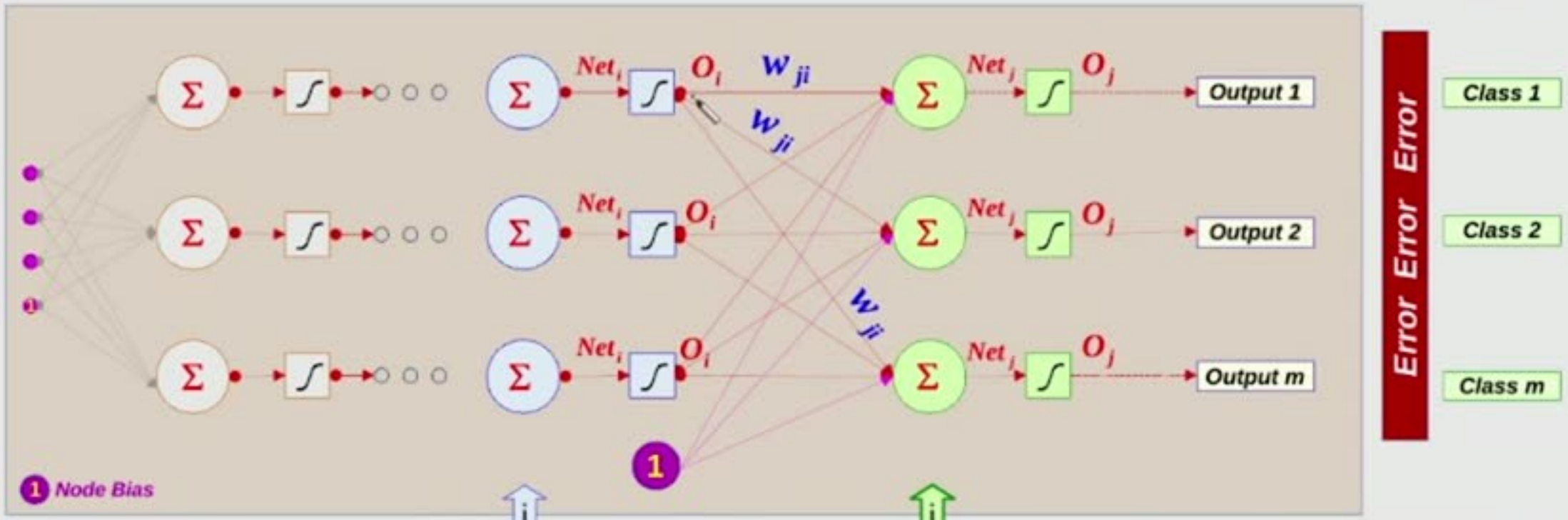
momaher@zewailcity.edu.eg



Neural Network Model
(Each node consists of Summation + Activation)



Neural Network Model
(Each node consists of Summation + Activation)



For Layer "j"

$$Net_j = \sum_i (w_{ji} O_i)$$

$$O_j = f(Net_j) = \frac{1}{1+e^{-Net_j}}$$

If Layer "j" is Output Layer:

$$E = \frac{1}{2} \sum_j (t_j - O_j)^2$$

Remember: Derivative of Sigmoid Function

$$O_j = f(Net_j) = \frac{1}{(1+e^{-Net_j})}$$



$$f(x) = \frac{1}{(1+e^{-x})}$$



$$f'(Net_j) = \frac{+\ e^{-Net_j}}{(1+e^{-Net_j})(1+e^{-Net_j})} = \frac{1*(1+\ e^{-Net_j}\ -1)}{(1+e^{-Net_j})(1+e^{-Net_j})}$$

$$f'(Net_j) = \frac{1}{(1+e^{-Net_j})}\left(\frac{(1+e^{-Net_j}-1)}{(1+e^{-Net_j})}\right)$$

$$= \frac{1}{(1+e^{-Net_j})}\left(\frac{(1+e^{-Net_j})}{(1+e^{-Net_j})}\ \frac{-1}{(1+e^{-Net_j})}\right)$$

$$f'(Net_j) = \frac{1}{(1+e^{-Net_j})}\left(1-\frac{1}{(1+e^{-Net_j})}\right)$$

$$\Rightarrow \quad f'(Net_j) = O_j(1-O_j)$$

Revision

$b = f(a)\ \ c = f(b)\ \Rightarrow\ c = f(a)$ *if a is changed , c will be chnaged accordingly)*

$$\frac{\partial c}{\partial a} = \frac{\partial c}{\partial b} * \frac{\partial b}{\partial a}$$ *Chain Rule*

$= f(\underline{W}) = w_1 . a_1 + w_2 . a_2 + w_3 . a_3 + \ldots\ldots + w_n . a_n = \sum_{i=1}^{n} w_i . a_i$

$$\frac{\partial b}{\partial w_1} = a_1 \quad \frac{\partial b}{\partial w_2} = a_2 \quad \frac{\partial b}{\partial w_i} = a_i$$

Conclusion: $b = f(\underline{W}) = \sum_{i=1}^{n} a_i w_i$ then $\frac{\partial b}{\partial w_i} = a_i$ *One Term ONLY*

$c = f(a_1), c = f(a_2), c = f(a_3)$ and $a_1 = f(b), a_2 = f(b), a_3 = f(b)$

$c = f(b) \Rightarrow$ if b is changed, c will be changed accordingly *[through* $a_1, a_2, a_3]$

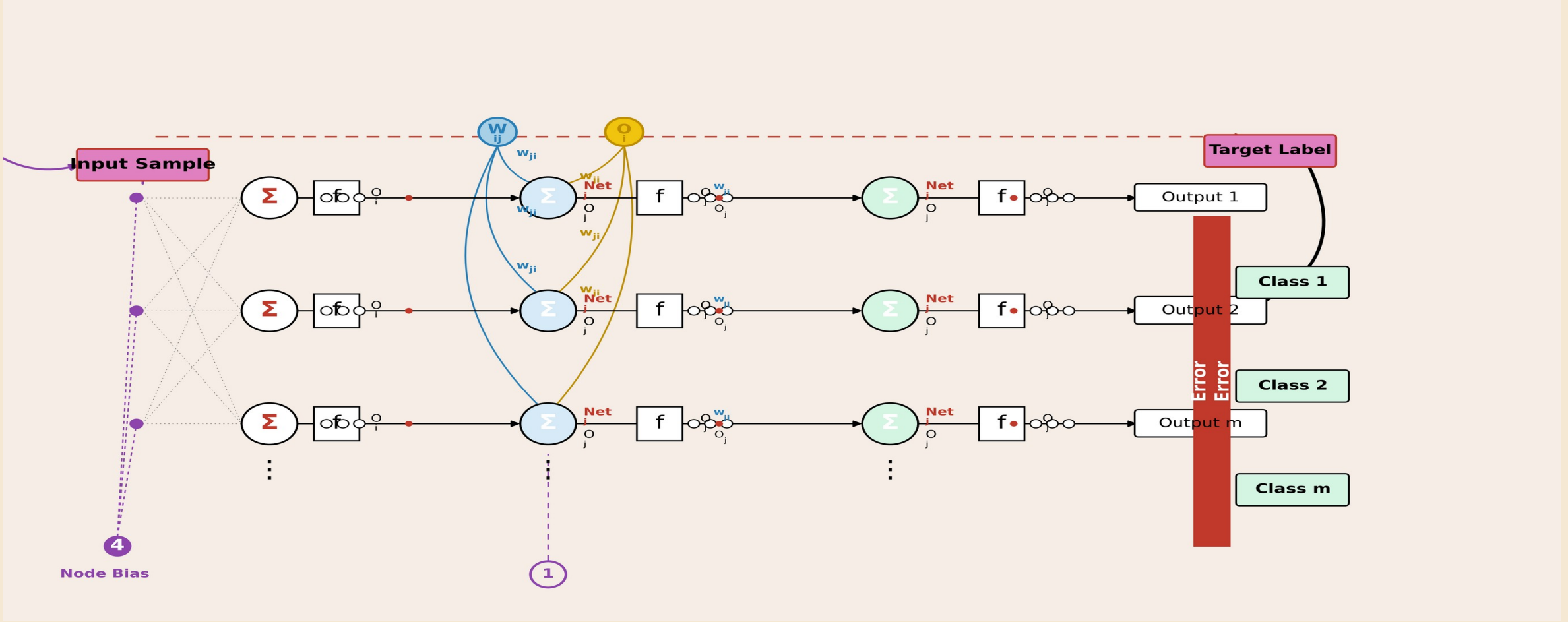
$$\frac{\partial c}{\partial b} = \left[\frac{\partial c}{\partial a_1} * \frac{\partial a_1}{\partial b}\right] + \left[\frac{\partial c}{\partial a_2} * \frac{\partial a_2}{\partial b}\right] + \left[\frac{\partial c}{\partial a_3} * \frac{\partial a_3}{\partial b}\right] = \sum_{i=1}^{3} \frac{\partial c}{\partial a_i} * \frac{\partial a_i}{\partial b}$$

Update Weights of Output Layer

(Weights from previous layer to output layer)

Neural Network Model

(Weights of Output Layer AFFECTS Calculated Error)



For Layer "j"

$$Net_j = \sum w_{ji} O_i$$

$$O_j = f(Net_j) = \frac{1}{1 + e^{-Net_j}}$$

↑

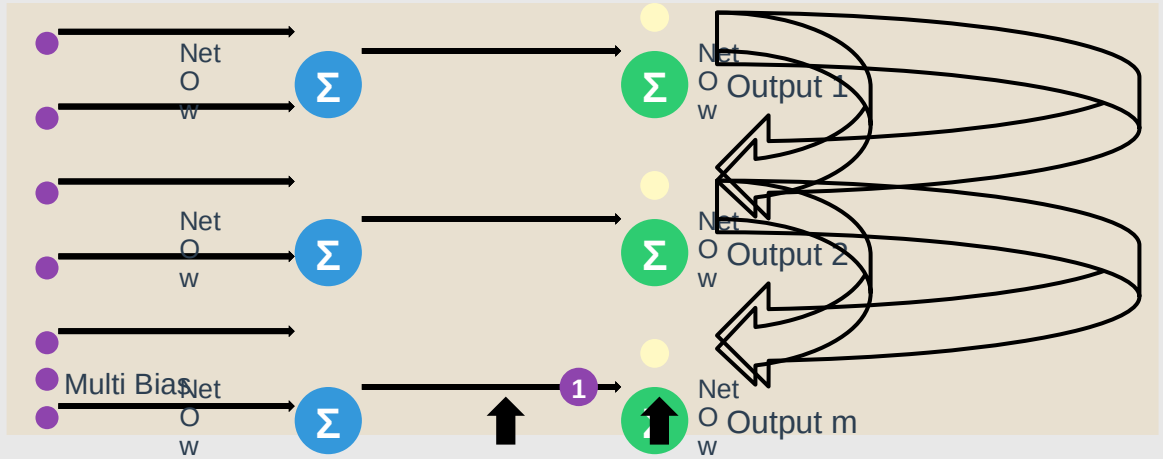
Previous Layer Layer (0)

↑

Current Layer is Output Layer (You are here)

If Layer "j" is Output Layer:

$$E = \frac{1}{2} \sum (t_j - O_j)^2$$

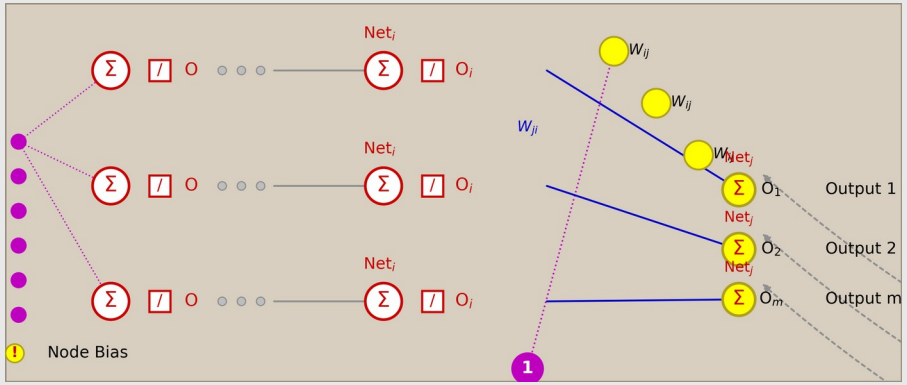

Net
O
w
Σ
Net
O
w
Output 1
Net
O
w
Σ
Net
O
w
Output 2
Multi Bias
1
Net
O
w
Σ
Net
O
w
Output m
Error Class 1
Error Class 2
Error Class m
Error
?

Gradient Descent

$$W_{ji}^{Updated} = W_{ji}^{old} - \eta \frac{\partial E}{\partial W_{ji}}$$

$$W_{ji}^{Updated} = W_{ji}^{old} + \Delta_{ji}$$

$$\text{Where } \Delta_{ji} = -\eta \frac{\partial E}{\partial W_{ji}}$$



Error

Error Class 1

Error Class 2

Error Class m

$$E = \frac{1}{2}\sum_j (t_j - O_j)^2$$

For Sigmoid Activation $O_j = f(Net_j) = \frac{1}{1+e^{-Net_j}}$

$$Net_j = \sum_i (w_{ji} O_i)$$

$$\frac{\partial E}{\partial O_j} = -(t_j - O_j)$$

$$\frac{\partial O_j}{\partial Net_j} = O_j(1 - O_j)$$

$$\frac{\partial Net_j}{\partial W_{ji}} = O_i$$

$$\frac{\partial E}{\partial W_{ji}} = \frac{\partial E}{\partial O_j} * \frac{\partial O_j}{\partial Net_j} * \frac{\partial Net_j}{\partial W_{ji}}$$

$$= -(t_j - O_j) * O_j(1 - O_j) * O_i$$

$$\frac{\partial E}{\partial O_j} = -(t_j - O_j)$$

$$\frac{\partial O_j}{\partial Net_j} = O_j(1 - O_j)$$

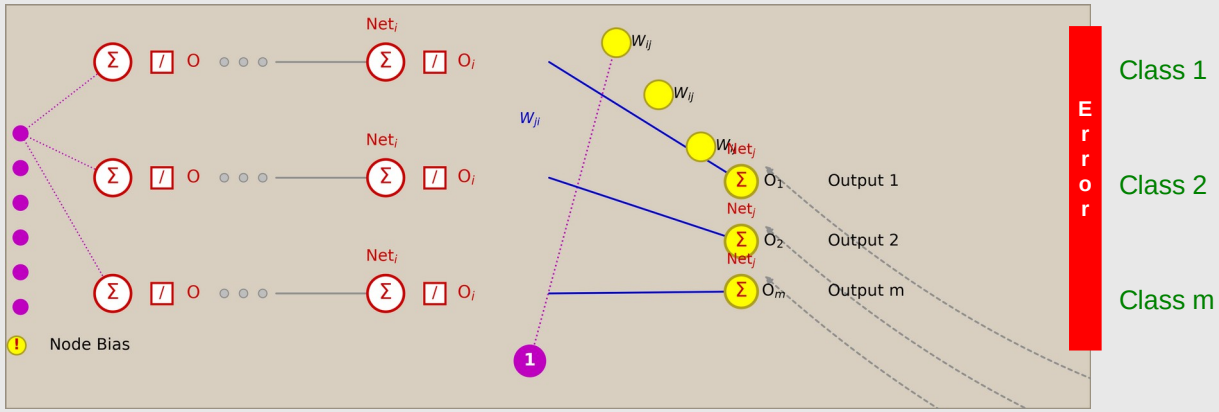
$$\frac{\partial Net_j}{\partial W_{ji}} = O_i$$

Gradient Descent

$$W_{ji}^{updated} = W_{ji}^{old} - \eta \frac{\partial E}{\partial W_{ji}}$$

$$W_{ji}^{updated} = W_{ji}^{old} + \Delta_{ji}$$

Where $\Delta_{ji} = -\eta \frac{\partial E}{\partial W_{ji}}$



$$E = \frac{1}{2}\sum_j (t_j - O_j)^2$$

For Sigmoid Activation

$$O_j = f(Net_j) = \frac{1}{1 + e^{-Net_j}}$$

$$Net_j = \sum_i (w_{ji} O_i)$$

$$\frac{\partial E}{\partial O_j} = -(t_j - O_j)$$

$$\frac{\partial O_j}{\partial Net_j} = O_j(1 - O_j)$$

$$\frac{\partial Net_j}{\partial W_{ji}} = O_i$$

$$\frac{\partial E}{\partial W_{ji}} = \frac{\partial E}{\partial O_j} \cdot \frac{\partial O_j}{\partial Net_j} \cdot \frac{\partial Net_j}{\partial W_{ji}} = \frac{\partial E}{\partial Net_j} \cdot \frac{\partial Net_j}{\partial W_{ji}}$$

$$= -(t_j - O_j) \cdot O_j(1 - O_j) \cdot O_i = -\delta_j \cdot O_i$$

Where $\delta_j = -\frac{\partial E}{\partial Net_j} = (t_j - O_j) \cdot O_j(1 - O_j)$ (δ_j depends on j ONLY)

$$\frac{\partial E}{\partial O_j} = -(t_j - O_j)$$

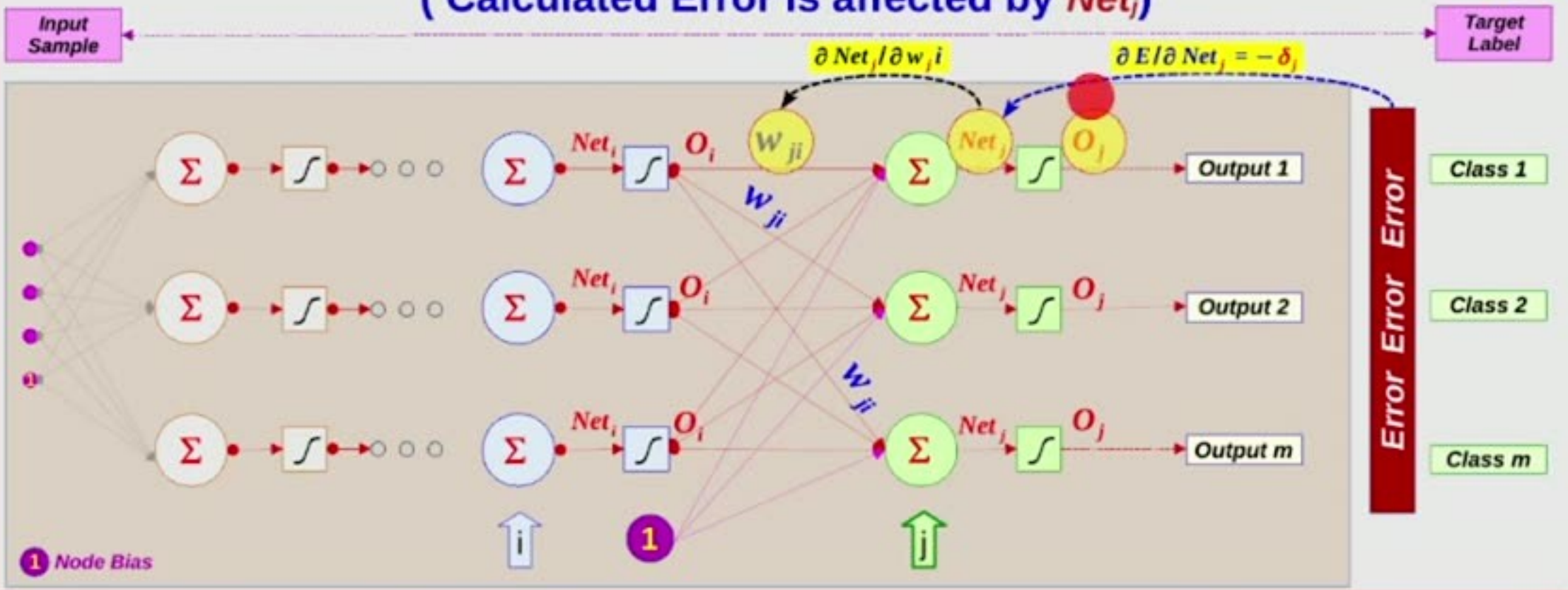
$$\frac{\partial O_j}{\partial Net_j} = O_j(1 - O_j)$$

$$\frac{\partial Net_j}{\partial W_{ji}} = O_i$$

$$\Delta_{ji} = -\eta \frac{\partial E}{\partial W_{ji}} = \eta \delta_j \cdot O_i$$ **Where** $\delta_j = (t_j - O_j) \cdot O_j(1 - O_j)$ for output Layer

Updated Weights of Output Layer

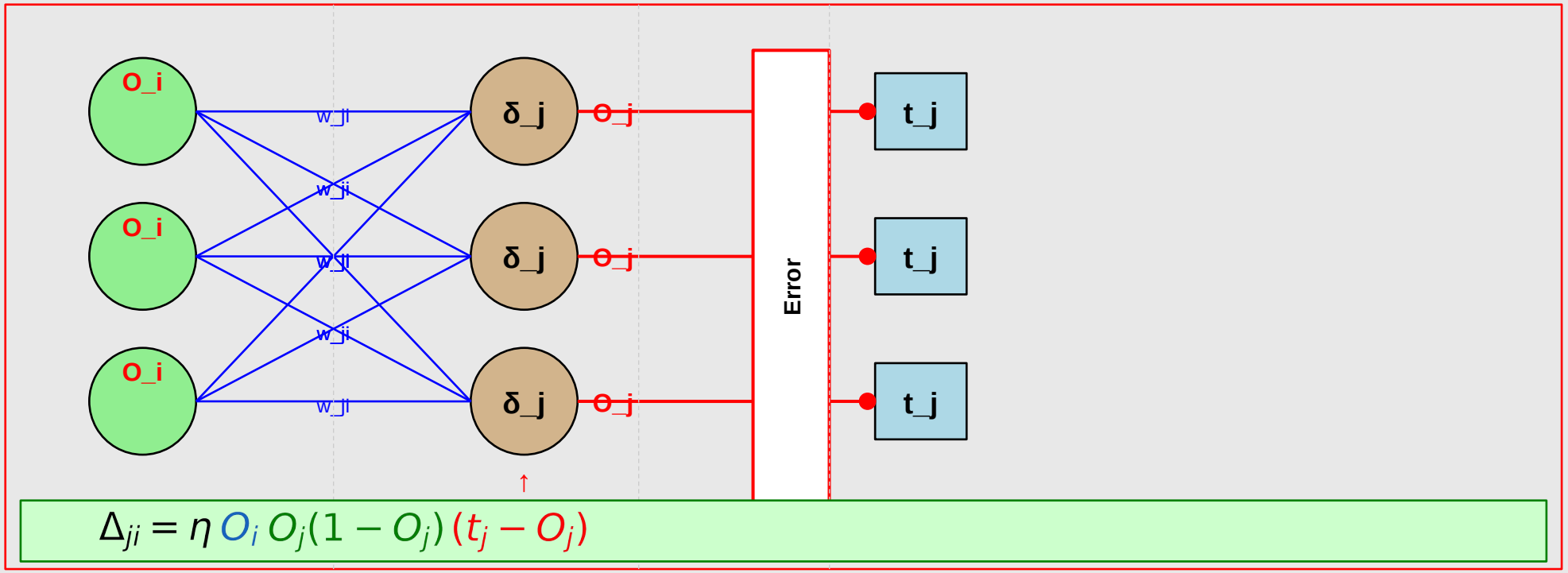
(Calculated Error is affected by Net_j)



$$\Delta_{ji} = -\eta \frac{\partial E}{\partial w_{ji}} = -\frac{\partial E}{\partial Net_j}\frac{\partial Net_j}{\partial w_{ji}} = \eta\ \delta_j\ *\ O_i \qquad \text{Where} \quad \delta_j = -\frac{\partial E}{\partial Net_j} \quad (\text{one } \delta \text{ for each Node } J)$$

For Output Layer, Sigmoid Activation $\Rightarrow$ $\delta_j = (t_j - O_j)\ * O_j(1-O_j) \Rightarrow$ $\Delta_{ji} = \eta\ (t_j - O_j)\ * O_j(1-O_j)\ *\ O_i$

At Output Layer



$\Delta_{ji} = \eta\, O_i\, \delta_j$

Where: $\delta_j = O_j(1 - O_j)\,(t_j - O_j)$

Update Weights of Hidden Layer

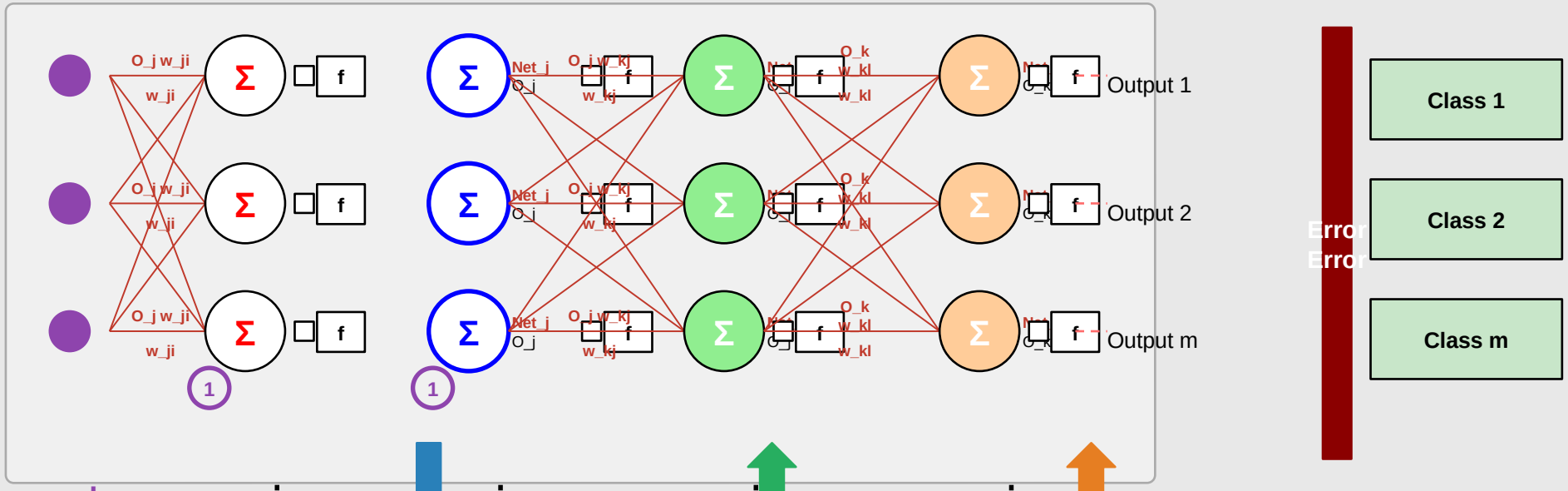
(Weights from previous layer to Hidden layer)

Neural Network Model

(Current Layer is Hidden Layer)

Input Sample

Target Label



For Layer "j"

$$Net_j = \Sigma_i(w_{ji} \cdot O_i)$$

$$O_j = f(Net_j) = \frac{1}{1 + e^{-Net_j}}$$

Previous Layer
(I)

Current Layer
(You are here)

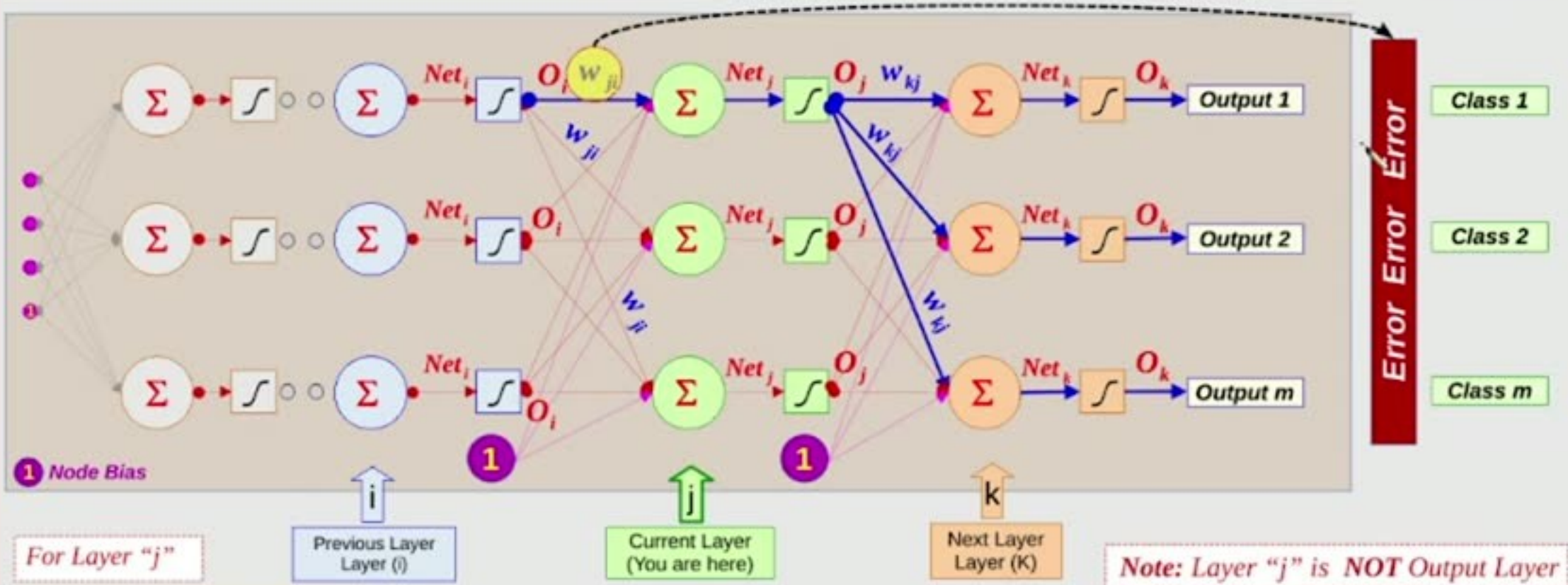
Next Layer
(K)

Note: Layer "j" is **NOT** Output Layer

$$E = \frac{1}{2}\Sigma_j(t_j - O_j)^2$$

X X X

Neural Network Model
(Weights of hidden layer AFFECT Calculated Error)



For Layer "j"

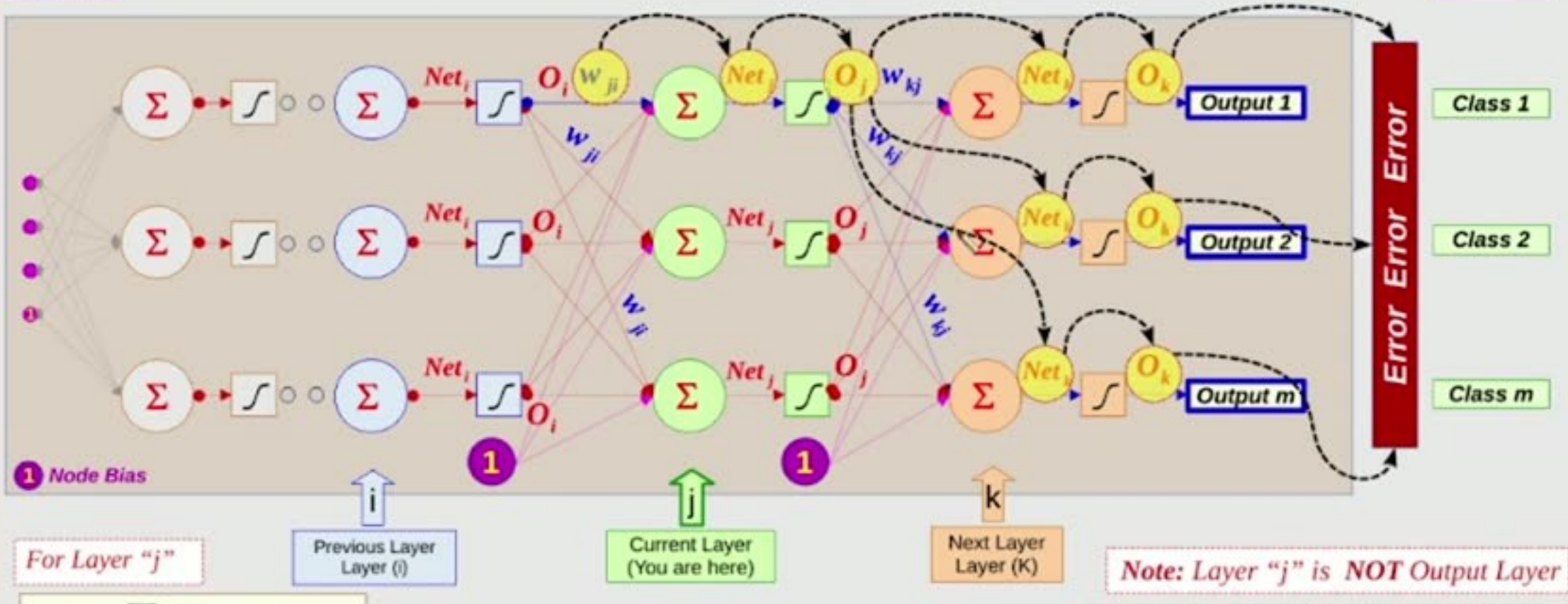
$$Net_j = \sum_i (w_{ji} O_i)$$

$$O_j = f(Net_j) = \frac{1}{1+e^{-Net_j}}$$

Note: Layer "j" is **NOT** Output Layer

$$E = \frac{1}{2} \sum_j (t_j - O_j)^2$$ XXX

Neural Network Model
(How Weights of hidden layer affect Calculated Error)



For Layer "j"

$$Net_j = \sum_i (w_{ji} O_i)$$

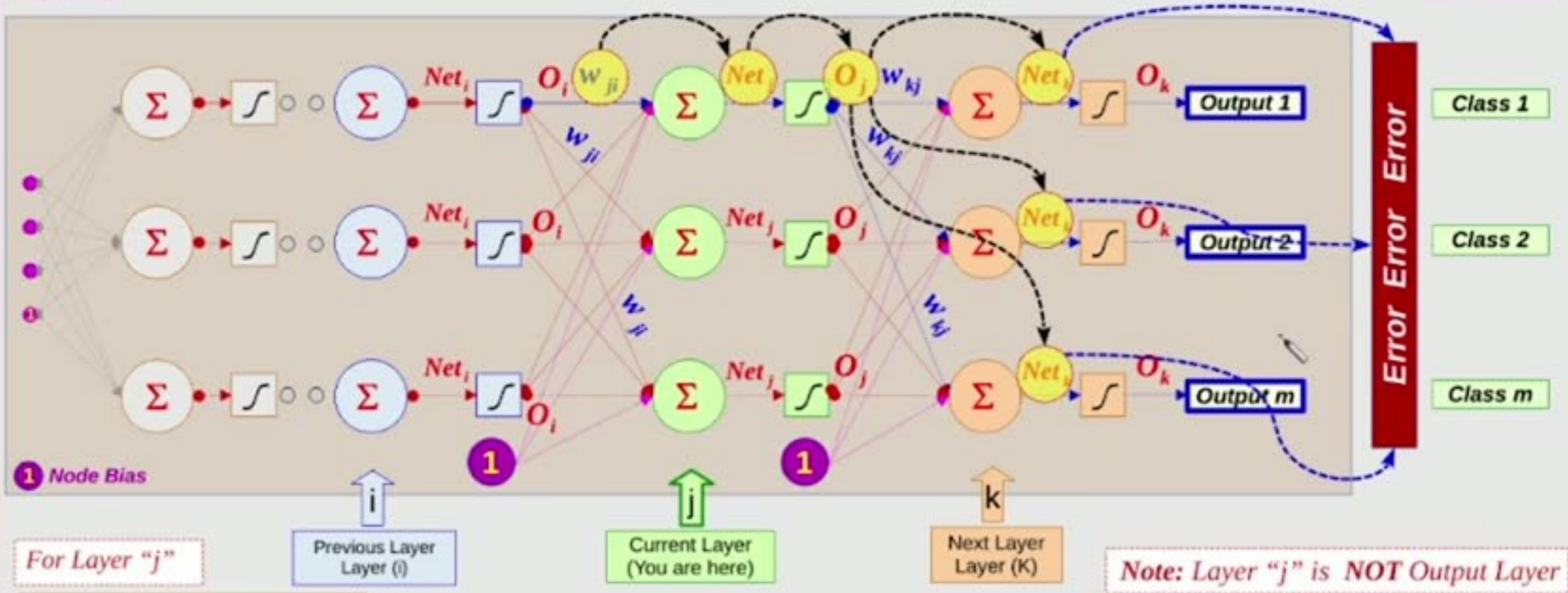
$$O_j = f(Net_j) = \frac{1}{1+e^{-Net_j}}$$

Note: *Layer "j" is* ***NOT*** *Output Layer*

$$E = \frac{1}{2} \sum_j (t_j - O_j)^2 \quad \text{XXX}$$

Neural Network Model

(Effect of Net_k on Calculated Error is known from O/P Layer)



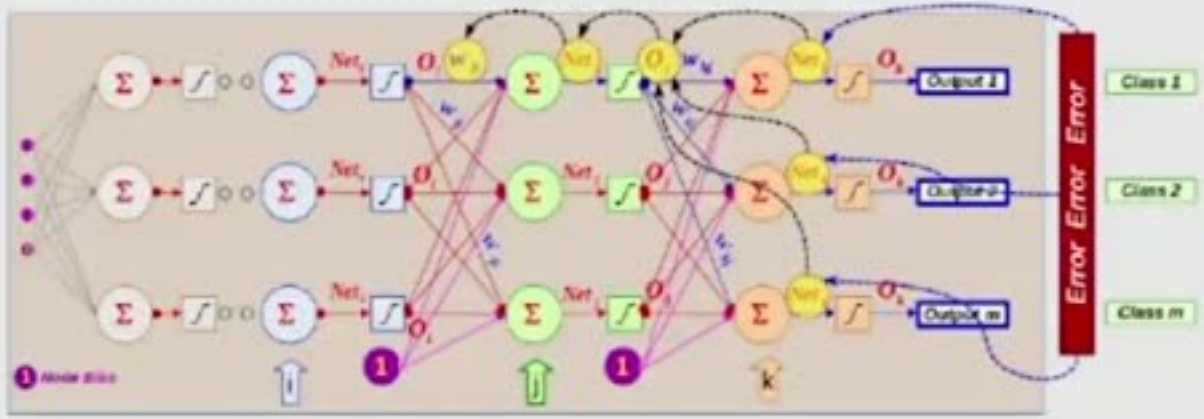
For Layer "j"

$$Net_j = \sum_i (w_{ji} O_i)$$

$$O_j = f(Net_j) = \frac{1}{1 + e^{-Net_j}}$$

Note: Layer "j" is **NOT** Output Layer

$$E = \frac{1}{2} \sum_j (t_j - O_j)^2$$ XXX



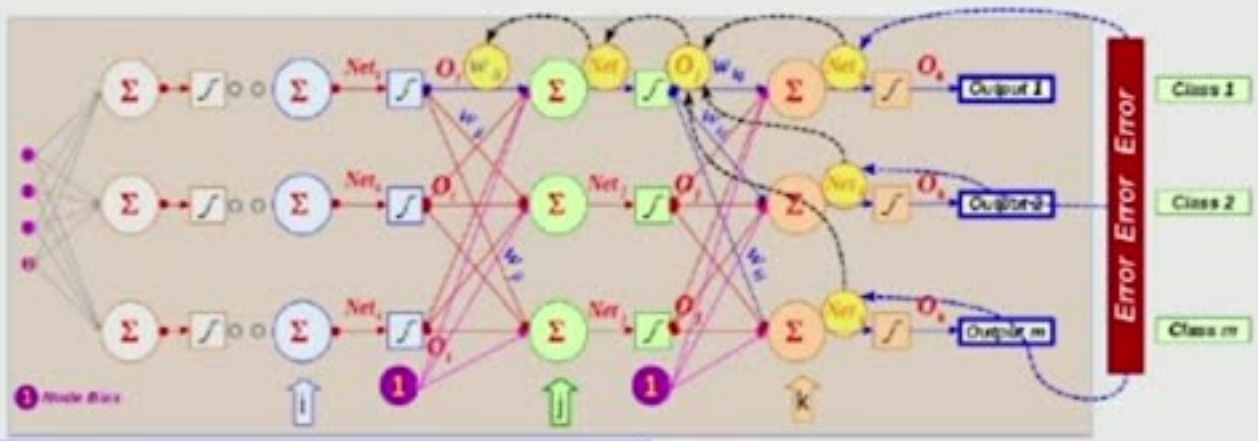
$$\frac{\partial E}{\partial w_{ji}} ?$$

W_{ji} Affects Net_j $\Rightarrow$

Net_j Affects O_j $\Rightarrow$

O_j Affects $(Net_{K=1}, Net_{k=2}, ..., Net_{k=m})$ $\Rightarrow$

$(Net_{K=1}, Net_{k=2}, ..., Net_{k=m})$ Affect $Error$



$$\frac{\partial E}{\partial w_{ji}} = \left[\frac{\partial E}{\partial Net_{k=1}} \frac{\partial Net_{k=1}}{\partial O_j} + \frac{\partial E}{\partial Net_{k=2}} \frac{\partial Net_{k=2}}{\partial O_j} + ... + \frac{\partial E}{\partial Net_{k=m}} \frac{\partial Net_{k=m}}{\partial O_j} \right] * \frac{\partial O_j}{\partial Net_j} * \frac{\partial Net_j}{\partial w_{ji}}$$

$$\frac{\partial E}{\partial Net_k} \Rightarrow -\delta_k$$

$$Net_k = \sum_k w_{kj} O_j \Rightarrow \frac{\partial Net_k}{\partial O_j} = w_{kj}$$

$$O_j = \frac{1}{1+e^{-Net_j}} \Rightarrow \frac{\partial O_j}{\partial Net_j} = O_j(1-O_j)$$

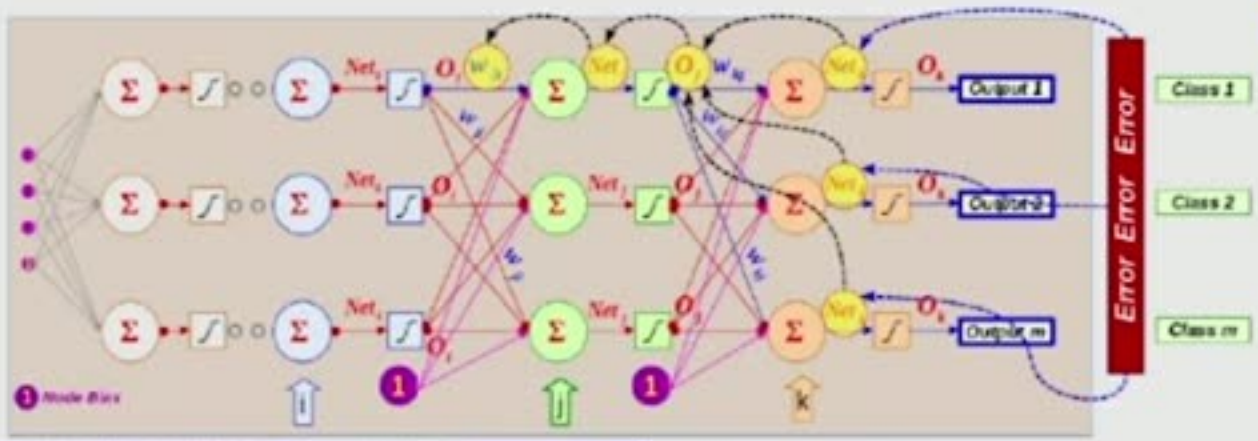
$$Net_j = \sum_j w_{ji} O_i \Rightarrow \frac{\partial Net_j}{\partial w_{ji}} = O_i$$

W_{ji} Affects Net_j $\Rightarrow$

Net_j Affects O_j $\Rightarrow$

O_j Affects $(Net_{K=1}, Net_{k=2}, ..., Net_{k=m})$ $\Rightarrow$

$(Net_{K=1}, Net_{k=2}, ..., Net_{k=m})$ Affect $Error$



$$\frac{\partial E}{\partial w_{ji}} = \left[\frac{\partial E}{\partial Net_{k=1}} \frac{\partial Net_{k=1}}{\partial O_j} + \frac{\partial E}{\partial Net_{k=2}} \frac{\partial Net_{k=2}}{\partial O_j} + ... + \frac{\partial E}{\partial Net_{k=m}} \frac{\partial Net_{k=m}}{\partial O_j} \right] * \frac{\partial O_j}{\partial Net_j} * \frac{\partial Net_j}{\partial w_{ji}}$$

$$\frac{\partial E}{\partial w_{ji}} = \underbrace{\left[\sum_{k=1}^{m} \frac{\partial E}{\partial Net_k} \frac{\partial Net_k}{\partial O_j} \right]}_{\partial E / \partial O_j} * \frac{\partial O_j}{\partial Net_j} * \frac{\partial Net_j}{\partial w_{ji}} = \underbrace{\left[\sum_{k=1}^{m} -\delta_k w_{kj} \right]}_{\partial E / \partial O_j} * \underbrace{O_j(1-O_j)}_{\partial E / \partial Net_j = -\delta_j} * O_i$$

$\partial E / \partial Net_j$ and $\partial E / \partial w_{ji}$

$$\Delta_{ji} = -\eta \frac{\partial E}{\partial W_{ji}} = \left[\sum_{k=1}^{m} \delta_k w_{kj} \right] * O_j(1-O_j) * O_i = \eta \ \delta_j \ * \ O_i$$

Where $\delta_j = \left[\sum_{k=1}^{m} \delta_k w_{kj} \right] * O_j(1-O_j)$ *(for each Node j at hidden layer)*

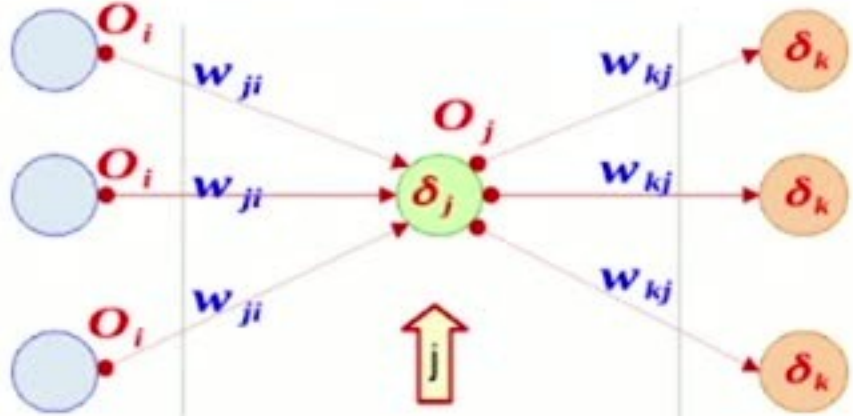
$$\frac{\partial E}{\partial Net_k} \Rightarrow -\delta_k$$

$$Net_k = \sum_k w_{kj} O_j \Rightarrow \frac{\partial Net_k}{\partial O_j} = w_{kj}$$

$$O_j = \frac{1}{1+e^{-Net_j}} \Rightarrow \frac{\partial O_j}{\partial Net_j} = O_j(1-O_j)$$

$$Net_j = \sum_j w_{ji} O_i \Rightarrow \frac{\partial Net_j}{\partial w_{ji}} = O_i$$

At Hidden Layer

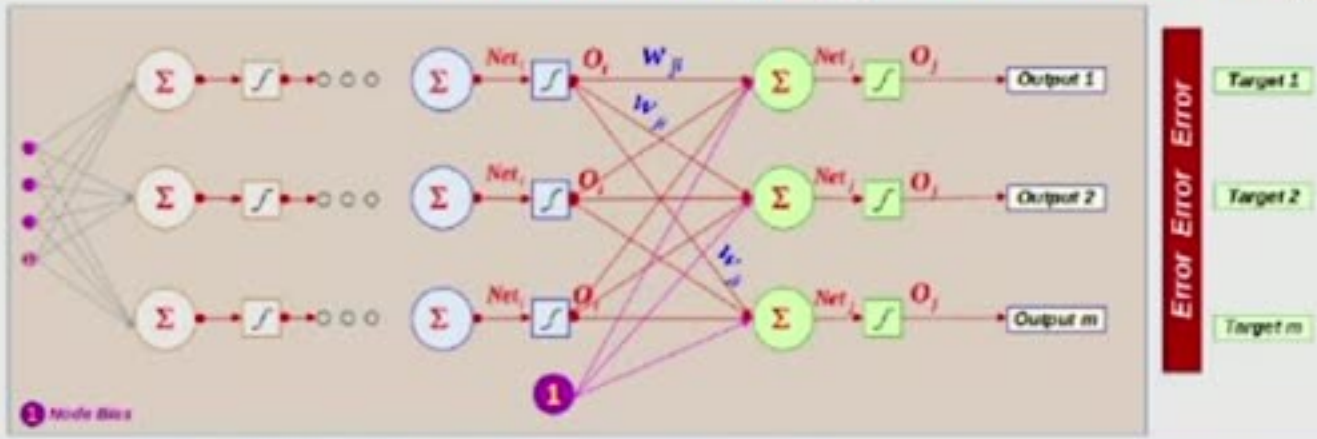


$$\Delta_{ji} = \eta \quad O_i \quad O_j(1-O_j) \quad [\sum_k (\delta_k w_{kj})]$$

$$\Delta_{ji} = \eta \quad O_i \quad \delta_j$$

Where: $\delta_j = O_j(1-O_j) \quad [\sum_k (\delta_k w_{kj})]$

Back Propagation Training Algorithm



[1] Initialize all Weights with random values
[2] Provide one training sample at a time (and its corresponding target label)

[3] *Go Forward* : Start from Input Layer, Calculate Net_j, O_j for each layer till reach output layer

[4] *Go Backward* : Start from Output Layer, Calculate Δ_{ji}, W_{ji} for each weights of ***Output*** layer

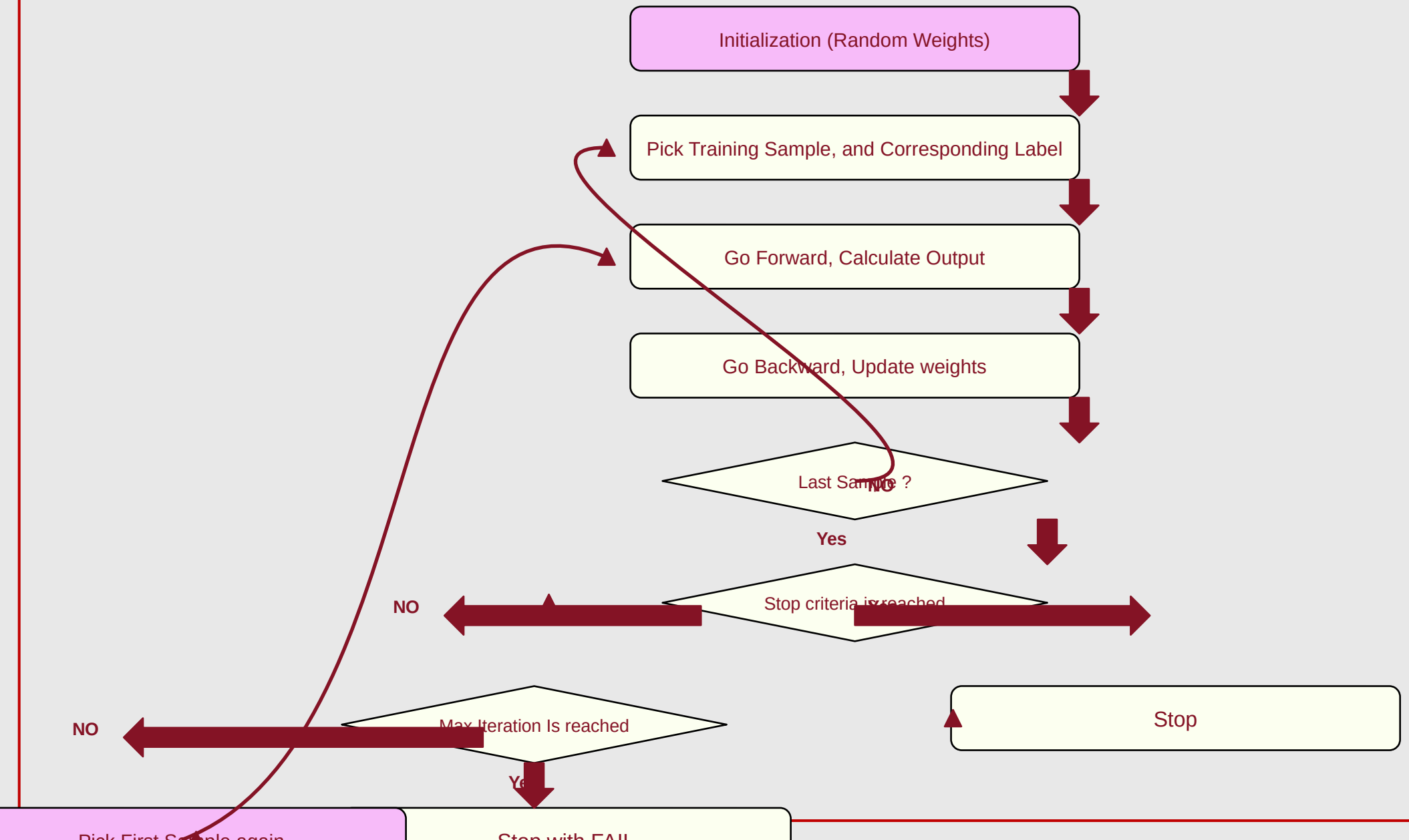
$\Delta_{ji} = \eta \; O_i \; O_j(1-O_j) \; (t_j - O_j)$ **Store:** $\delta_j = O_j(1-O_j) \; (t_j - O_j)$ *(for each node at output Layer)*

[5] *Go Backward* : go to previous Layer, Calculate Δ_{ji}, W_{ji} for each weights of ***Hidden*** layer

$\Delta_{ji} = \eta \; O_i \; O_j(1-O_j) \; [\sum_k (\delta_k w_{kj})]$ **Store:** $\delta_j = O_j(1-O_j) \; [\sum_k (\delta_k w_{kj})]$ *(for each node at hidden Layer)*

[6] *Repeat* **[5]** *till reach first hidden layer*

Remember: $\delta_k = \delta_j$ of next layer (which is already calulated in previous step)


Initialization (Random Weights)
Pick Training Sample, and Corresponding Label
Go Forward, Calculate Output
Go Backward, Update weights
Last Sample ?
NO
Yes
NO
Stop criteria is reached
NO
Max Iteration Is reached
Stop

Stopping Criteria

[1] Total Error = zero (Theoretical)

[2] Total Error < certain value (Threshold)

[3] Percentage Change of Weights is less than certain value (Threshold)

[4] Reach Maximum number of iterations (Epochs) without satisfying [2] or [3]

Stopping is due to reaching Maximum number of iterations = Training is FAILED

Expected Reasons:

[1] The defined Maximum Number of Epochs is small. Try to increase the limit

[2] Under-fitting (Training samples are not adequate). Try to provide more training samples

[3] Model Design is not suitable to training data. Try to add more nodes in hidden layer or add more hidden Layers

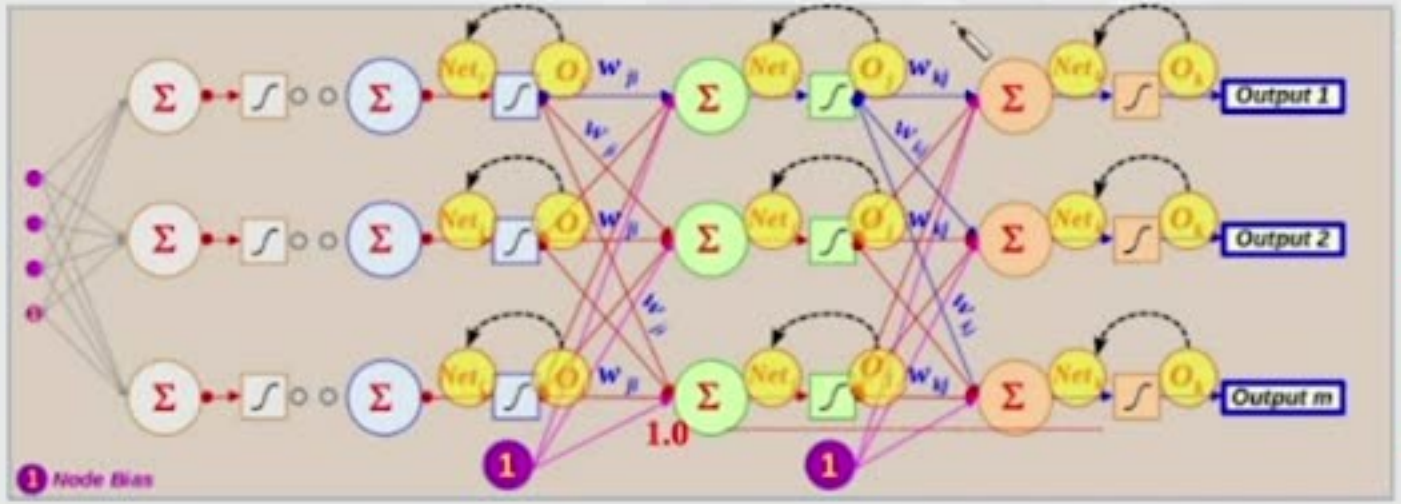
[4] Data is NOT separable by Neural Network. Try Another Models (e.g. SVM with Kernels)

Vanishing and Exploding Gradients

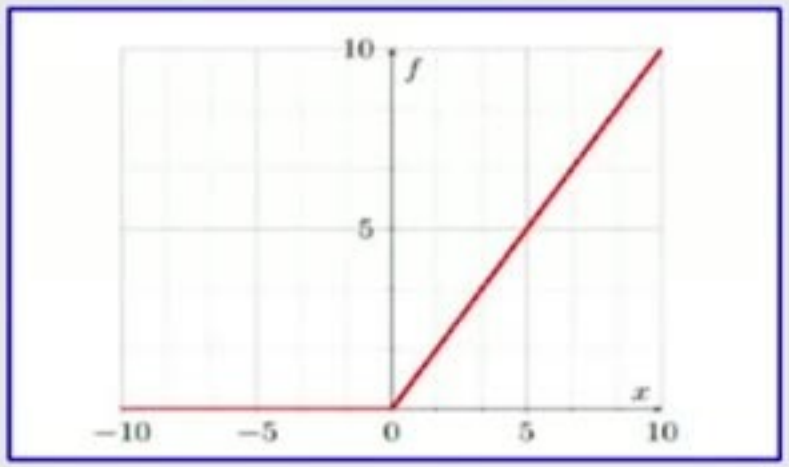
Long Chain of Derivatives

$$f(x) = ReLU = \begin{cases} 0 & for\ x < 0 \\ x & for\ x \geqslant 0 \end{cases}$$

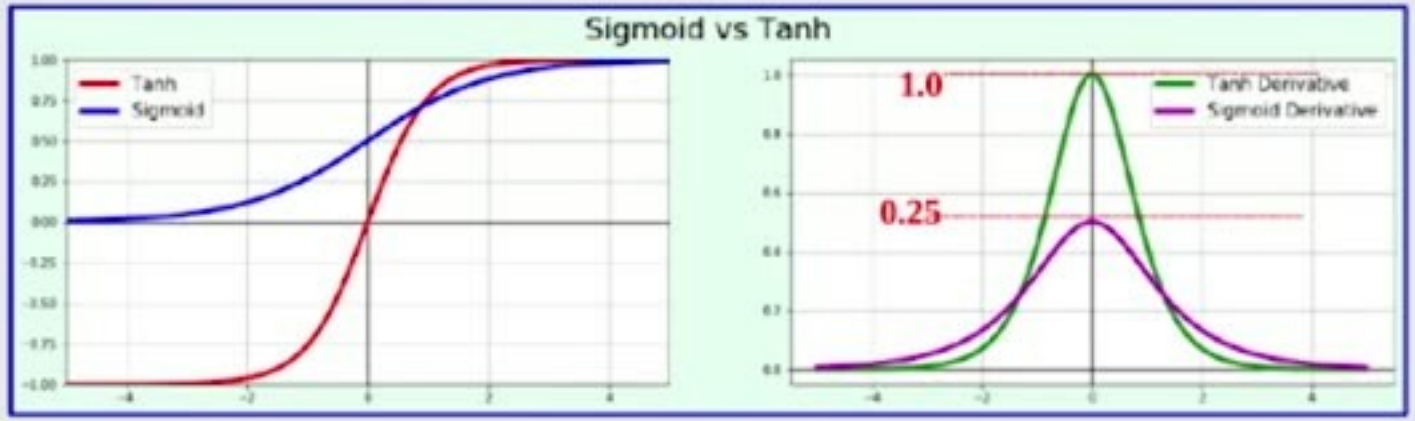
$$f'(x) = \begin{cases} 0 & for\ x < 0 \\ 1 & for\ x \geqslant 0 \end{cases}$$



Exploding Gradients



Vanishing Gradients

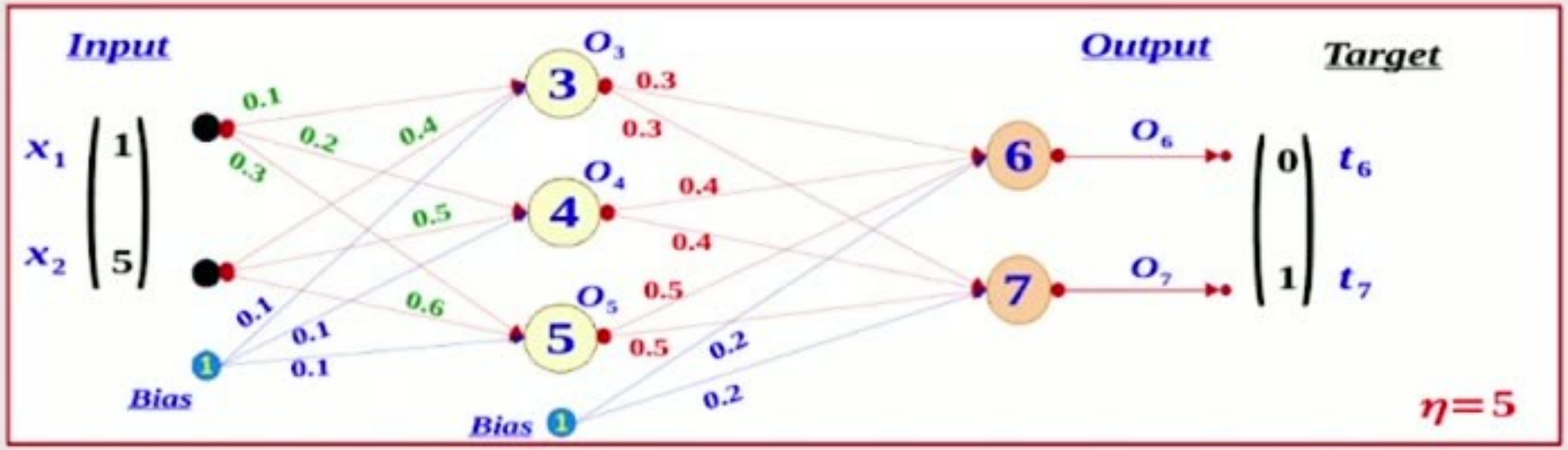


Solved Example

[Update Weights of Output Layer and Hidden Layer]

Using one training sample

Example



Forward

$$Net_3 = 1*0.1 + 5*0.4 + 1*0.1 = 2.2$$

$$Net_4 = 1*0.2 + 5*0.5 + 1*0.1 = 2.8$$

$$Net_5 = 1*0.3 + 5*0.6 + 1*0.1 = 3.4$$

$$O_3 = \frac{1}{1+e^{-2.2}} = 0.9$$

$$O_4 = \frac{1}{1+e^{-2.8}} = 0.943$$

$$O_5 = \frac{1}{1+e^{-3.4}} = 0.968$$

$$Net_6 = 0.9*0.3 + 0.943*0.4 + 0.968*0.5 + 1*0.2 = 1.3312$$

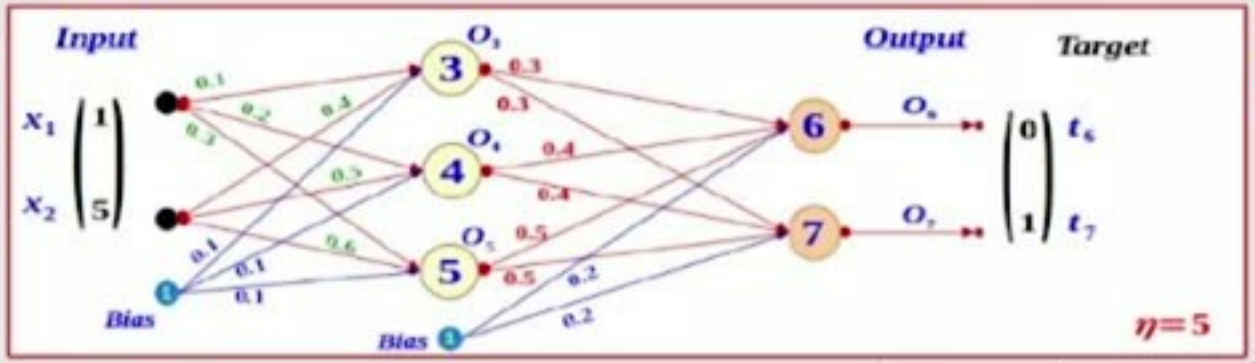
$$Net_6 = 0.9*0.3 + 0.943*0.4 + 0.968*0.5 + 1*0.2 = 1.3312$$

$$O_6 = \frac{1}{1+e^{-1.3312}} = 0.791$$

$$O_7 = \frac{1}{1+e^{-1.3312}} = 0.791$$

$$\delta_j = O_j(1-O_j)\,(t_j-O_j)$$

$$\Delta_{ji} = \eta\ O_i\ O_j(1-O_j)\ (t_j-O_j) = \eta\ O_i\ \delta_j$$



Backward (Update Output Layer Weights)

$$\delta_6 = 0.791(1-0.791)(0-0.791) = -0.131$$
$$\delta_7 = 0.791(1-0.791)(1-0.791) = 0.035$$

$$O_3 = \frac{1}{1+e^{-2.2}} = 0.9$$
$$O_4 = \frac{1}{1+e^{-2.8}} = 0.943$$
$$O_5 = \frac{1}{1+e^{-3.4}} = 0.968$$

$$\Delta W_{63} = 5*(-0.131)*0.9 = -0.59$$
$$\Delta W_{64} = 5*(-0.131)*0.943 = -0.618$$
$$\Delta W_{65} = 5*(-0.131)*0.968 = -0.634$$
$$\Delta W_{6b} = 5*(-0.131)*1.0 = -0.655$$

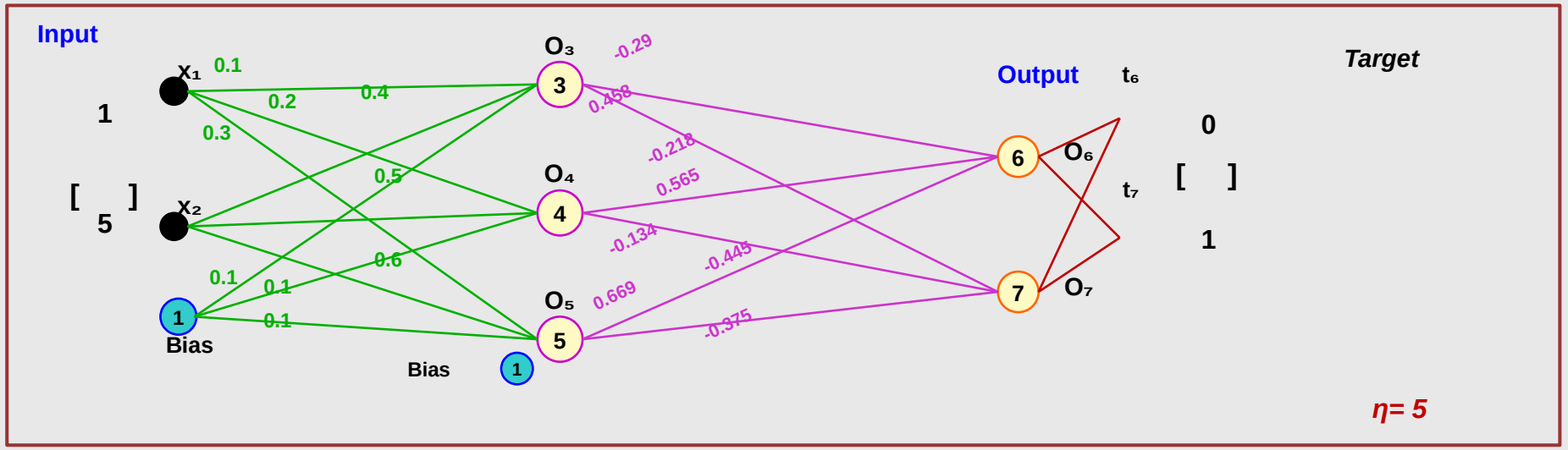
$$W_{63} = 0.3-0.59 = -0.29$$
$$W_{64} = 0.4-0.18 = -0.218$$
$$W_{65} = 0.5-0.634 = -0.134$$
$$W_{6b} = 0.2-0.655 = -0.455$$

$$O_6 = \frac{1}{1+e^{-1.3312}} = 0.791$$
$$O_7 = \frac{1}{1+e^{-1.3312}} = 0.791$$

$$\Delta W_{73} = 5*(0.035)*0.9 = 0.158$$
$$\Delta W_{74} = 5*(0.035)*0.943 = 0.165$$
$$\Delta W_{75} = 5*(0.035)*0.968 = 0.169$$
$$\Delta W_{7b} = 5*(0.035)*1.0 = 0.175$$

$$W_{73} = 0.3+0.158 = 0.458$$
$$W_{74} = 0.4+0.165 = 0.565$$
$$W_{75} = 0.5+0.169 = 0.669$$
$$W_{7b} = 0.2+0.175 = 0.375$$

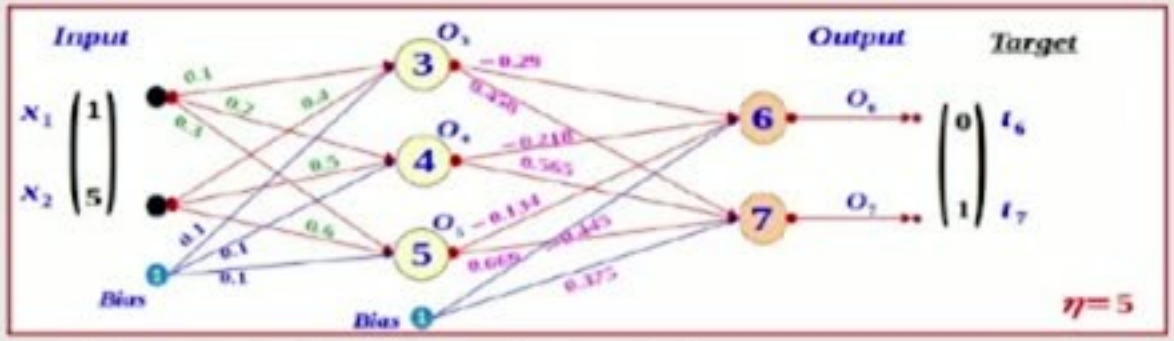
After Updating O/p Layer weights



Example

$$\delta_j = O_j(1-O_j)\ [\sum_k (\delta_k w_{kj})]$$

$$\Delta_{ji} = \eta\ O_i\ O_j(1-O_j)\ [\sum_k (\delta_k w_{kj})] = \eta\ O_i\ \delta_j$$



$\delta_6 = 0.791(1-0.791)(0-0.791) = -0.131$

$\delta_7 = 0.791(1-0.791)(1-0.791) = 0.035$

$$O_3 = \frac{1}{1+e^{-2.2}} = 0.9$$

$$O_4 = \frac{1}{1+e^{-2.8}} = 0.943$$

$$O_5 = \frac{1}{1+e^{-3.4}} = 0.968$$

Backward (Update *hidden* Layer Weights)

$\delta_3 = O_3(1-O_3)[\delta_6 * W_{63} + \delta_7 * W_{73}]$

$\delta_3 = 0.9(1-0.9)[-0.131 * -0.29 + 0.035 * 0.458] = 0.005$

$\delta_4 = 0.943(1-0.9943)[-0.131 * -0.218 + 0.035 * 0.565] = 0.003$

$\delta_5 = 0.968(1-0.968)[-0.131 * -0.134 + 0.035 * 0.669] = 0.001$

$$O_6 = \frac{1}{1+e^{-1.3312}} = 0.791$$

$$O_7 = \frac{1}{1+e^{-1.3312}} = 0.791$$

$\Delta W_{31} = 5 * 0.005 * 1 = 0.025$ → $W_{31} = 0.1 + 0.025 = 0.125$

$\Delta W_{32} = 5 * 0.005 * 5 = 0.125$ → $W_{32} = 0.4 + 0.125 = 0.525$

$\Delta W_{3b} = 5 * 0.005 * 1 = 0.025$ → $W_{3b} = 0.1 + 0.025 = 0.125$

$\Delta W_{41} = 5 * 0.003 * 1 = 0.015$ → $W_{41} = 0.2 + 0.015 = 0.215$

$\Delta W_{42} = 5 * 0.003 * 5 = 0.075$ → $W_{42} = 0.5 + 0.075 = 0.575$

$\Delta W_{4b} = 5 * 0.003 * 1 = 0.015$ → $W_{4b} = 0.1 + 0.015 = 0.115$

$\Delta W_{51} = 5 * 0.001 * 1 = 0.005$ → $W_{51} = 0.3 + 0.005 = 0.305$

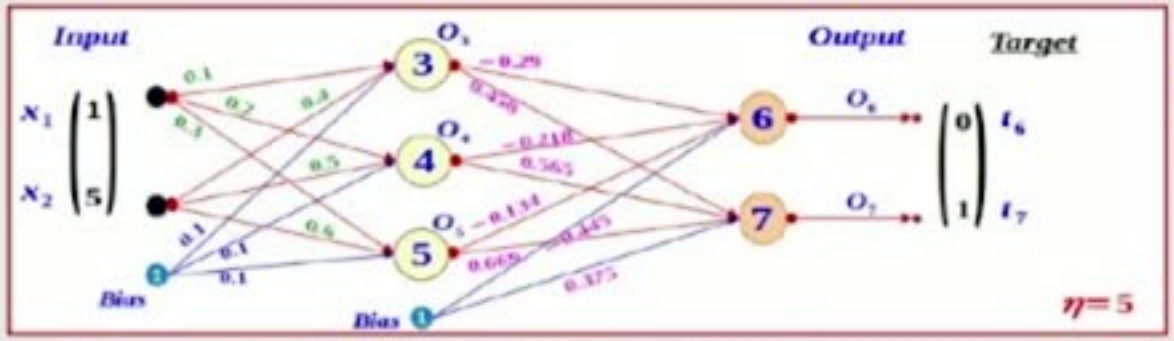
$\Delta W_{52} = 5 * 0.001 * 5 = 0.025$ → $W_{52} = 0.6 + 0.025 = 0.625$

$\Delta W_{5b} = 5 * 0.001 * 1 = 0.005$ → $W_{5b} = 0.1 + 0.005 = 0.105$

Example

$$\delta_j = O_j(1-O_j)\ [\sum_k (\delta_k w_{kj})]$$

$$\Delta_{ji} = \eta\ O_i\ O_j(1-O_j)\ [\sum_k (\delta_k w_{kj})] = \eta\ O_i\ \delta_j$$



$\delta_6 = 0.791(1-0.791)(0-0.791) = -0.131$

$\delta_7 = 0.791(1-0.791)(1-0.791) = 0.035$

$$O_3 = \frac{1}{1+e^{-2.2}} = 0.9$$

$$O_4 = \frac{1}{1+e^{-2.8}} = 0.943$$

$$O_5 = \frac{1}{1+e^{-3.4}} = 0.968$$

Backward (Update ***hidden*** Layer Weights)

$\delta_3 = O_3(1-O_3)[\delta_6 * W_{63} + \delta_7 * W_{73}]$

$\delta_3 = 0.9(1-0.9)[-0.131 * -0.29 + 0.035 * 0.458] = 0.005$

$\delta_4 = 0.943(1-0.9943)[-0.131 * -0.218 + 0.035 * 0.565] = 0.003$

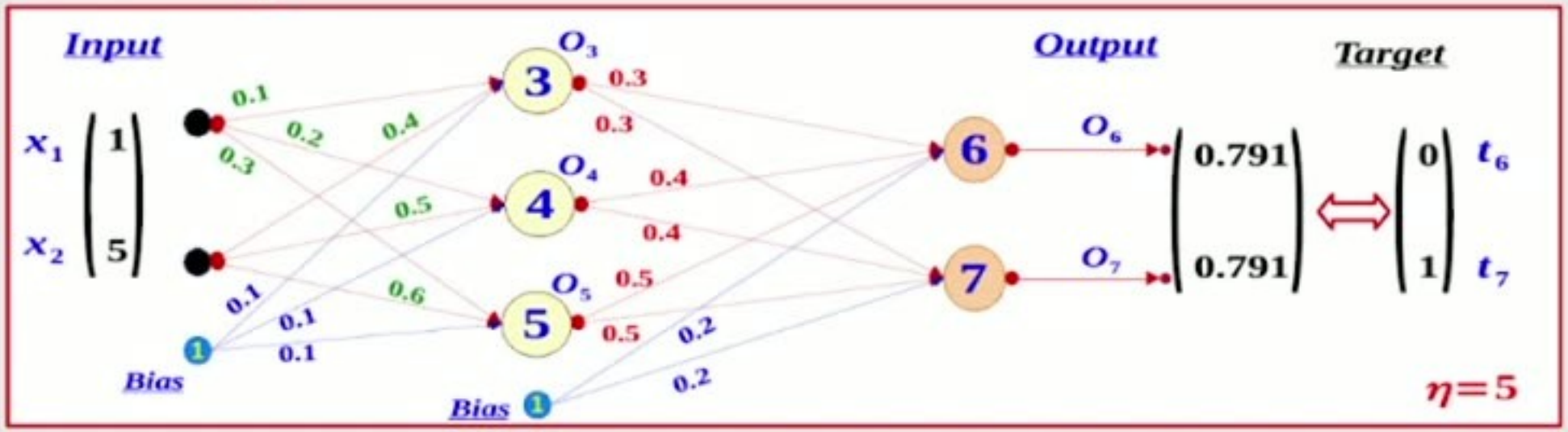
$\delta_5 = 0.968(1-0.968)[-0.131 * -0.134 + 0.035 * 0.669] = 0.001$

$$O_6 = \frac{1}{1+e^{-1.3312}} = 0.791$$

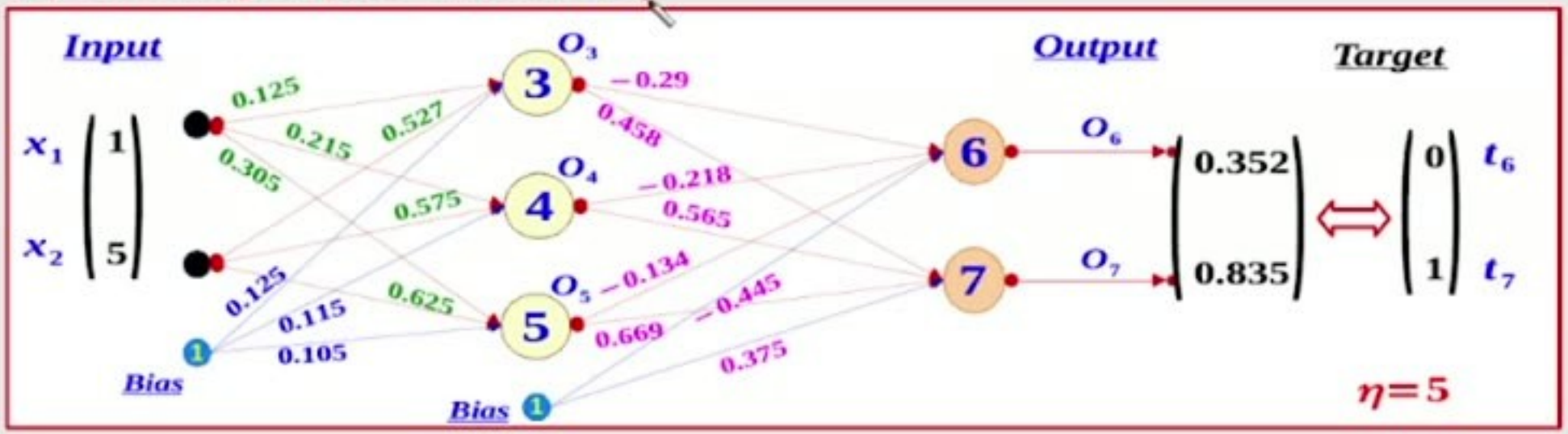
$$O_7 = \frac{1}{1+e^{-1.3312}} = 0.791$$

$\Delta W_{31} = 5 * 0.005 * 1 = 0.025$	$W_{31} = 0.1 + 0.025 = 0.125$
$\Delta W_{32} = 5 * 0.005 * 5 = 0.125$	$W_{32} = 0.4 + 0.125 = 0.525$
$\Delta W_{3b} = 5 * 0.005 * 1 = 0.025$	$W_{3b} = 0.1 + 0.025 = 0.125$
$\Delta W_{41} = 5 * 0.003 * 1 = 0.015$	$W_{41} = 0.2 + 0.015 = 0.215$
$\Delta W_{42} = 5 * 0.003 * 5 = 0.075$	$W_{42} = 0.5 + 0.075 = 0.575$
$\Delta W_{4b} = 5 * 0.003 * 1 = 0.015$	$W_{4b} = 0.1 + 0.015 = 0.115$
$\Delta W_{51} = 5 * 0.001 * 1 = 0.005$	$W_{51} = 0.3 + 0.005 = 0.305$
$\Delta W_{52} = 5 * 0.001 * 5 = 0.025$	$W_{52} = 0.6 + 0.025 = 0.625$
$\Delta W_{5b} = 5 * 0.001 * 1 = 0.005$	$W_{5b} = 0.1 + 0.005 = 0.105$

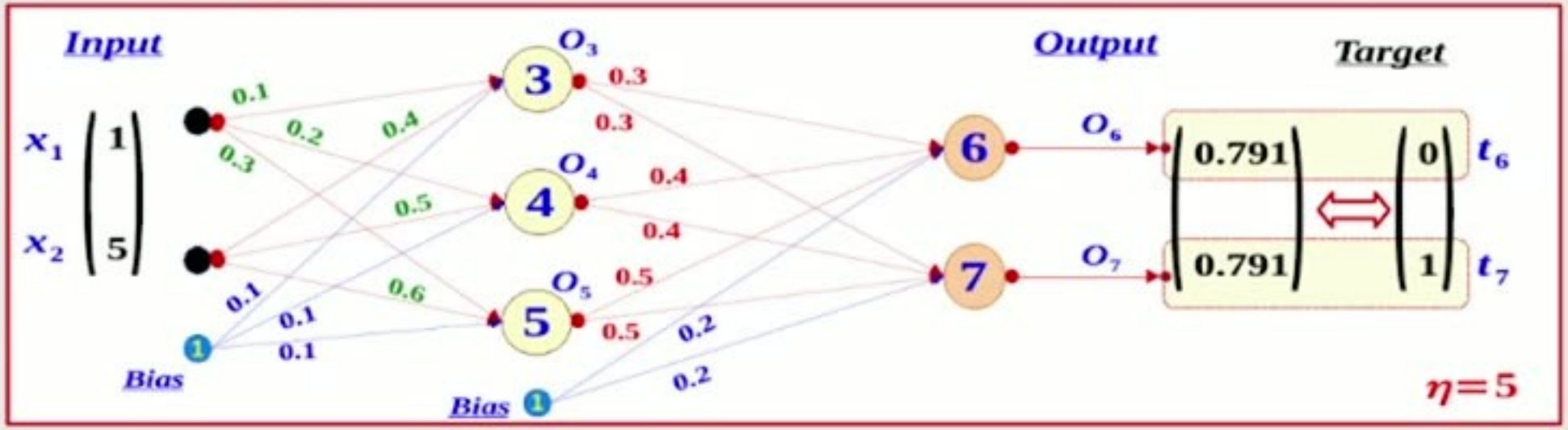
Initialized weights



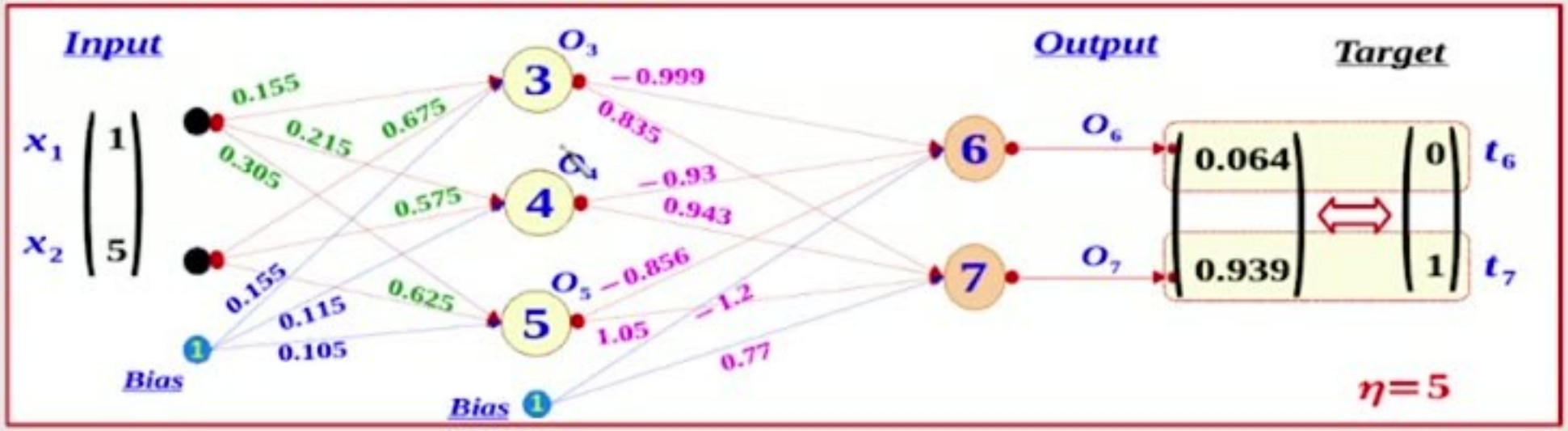
Updated weights (after iteration # 1)



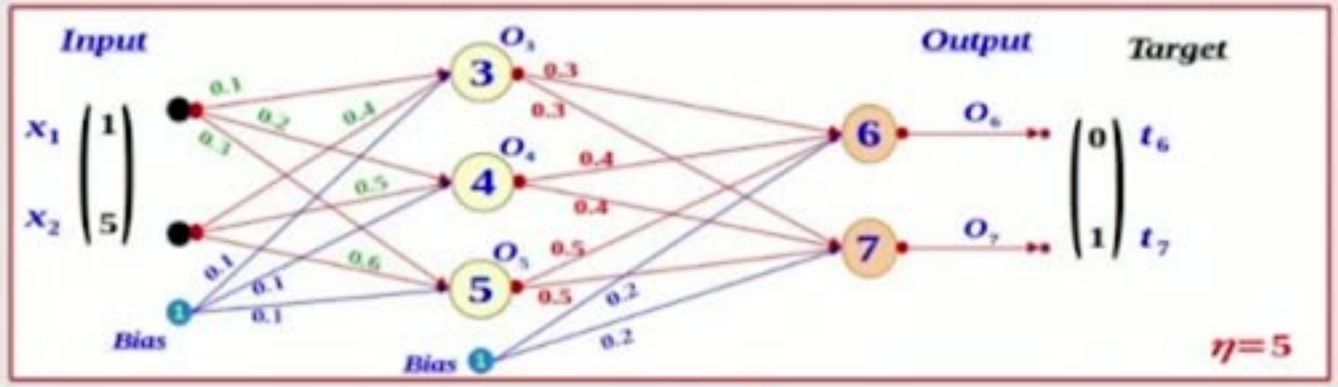
Initialized weights



Updated weights (after iteration # 10)



Forward Calculations



	O_6	O_7	Squared Error
Initial Weights	0.791	0.791	0.669362
Iteration 1	0.352	0.835	0.151129
Iteration 2	0.152	0.875	0.038729
Iteration 3	0.119	0.895	0.025186
Iteration 4	0.102	0.908	0.018868
Iteration 5	0.091	0.917	0.01517
	⋮	⋮	
Target	**0**	**1**	